

课程笔记

CS336 p05 GPU 与 TPU：硬件直觉与高效实现

Claude 课程笔记

2026 年 6 月 8 日



视频作者/频道：脑袋要有光

发布日期：2026-05-21

视频时长：1:18:39

视频链接：<https://www.bilibili.com/video/BV12gLx6xEcq?p=5>

目录

1 引言：为什么系统知识对语言建模至关重要	3
1.1 课程定位：从神经网络到系统模块	3
1.2 算力是语言模型扩展的核心驱动力	3
1.3 从 CPU 串行到 GPU/TPU 并行：后登纳德时代的算力增长	4
1.4 本章小结	5
2 GPU 硬件与编程模型	6
2.1 CPU vs. GPU：延迟 vs. 吞吐的设计理念	6
2.2 SM 与物理布局	7
2.3 存储层次：寄存器、共享内存/L1、L2、全局内存/HBM	8
2.4 CUDA 执行模型：线程、线程块、Warp 与 SIMD	9
2.4.1 线程 (Thread)	9
2.4.2 线程块 (Block / Thread Block)	10
2.4.3 线程束 (Warp)	10
2.4.4 CUDA 内存模型	10
2.5 本章小结	11
3 TPU 简介与 GPU/TPU 对比	12
3.1 为什么需要了解 TPU	12
3.2 TPU 的宏观结构：控制器、向量单元、MXU/脉动阵列	12
3.3 GPU 与 TPU 的“趋同演化”与关键差异	14
3.4 本章小结	15
4 让 GPU 跑得更快的六个技巧	16
4.1 屋顶线模型与计算强度	16
4.2 技巧 1：避免控制发散	17
4.3 技巧 2：低精度计算与量化	18
4.3.1 从 FP32 到 BF16	18
4.3.2 FP8 与块缩放格式	19
4.3.3 MXFP4	19
4.4 技巧 3：算子融合	20
4.5 技巧 4：重计算	21
4.6 技巧 5：合并内存访问与突发段	21
4.7 技巧 6：分块与波量化	22
4.7.1 分块矩阵乘法	22
4.7.2 波量化	23
4.7.3 整除性对吞吐量的影响	24
4.8 本章小结	25
5 综合案例：FlashAttention	26
5.1 问题背景	26
5.2 分块与在线 softmax	27

5.3	融合与重计算	29
5.4	本章小结	30
6	总结与延伸	31
6.1	主讲人的 closing message	31
6.2	本讲核心要点回顾	31
6.3	延伸思考	32

1 引言：为什么系统知识对语言建模至关重要

1.1 课程定位：从神经网络到系统模块

本讲是斯坦福 CS336《从零开始的语言建模》2026 年春季课程中“系统模块”(Systems) 的开篇。在前面的课程里，我们已经学习了神经网络的基础知识；从现在开始，我们将深入理解 GPU 并行化与推理，探究语言模型在实际硬件上运行的底层逻辑。

主讲人在开场时提到，系统知识是整个技术栈中“最能一步步推理清楚”的一层：它不像某些黑箱算法那样依赖直觉，而是可以通过清晰的逻辑链条得出可验证的结论。然而，第一次接触 GPU 的人往往会感到它既像魔法，又像某种难以捉摸的古怪装置。本节课的目标，就是揭开这层神秘面纱，让大家在课程结束时能够解释矩阵乘法吞吐量随维度呈现奇特起伏的“谜题”曲线——为什么它不是单调上升，而是会出现周期性的高峰与低谷。

为什么系统知识对非系统从业者同样重要

即使你不是系统专家，只要从事模型架构研究或性能分析，就必须了解模型运行在怎样的硬件抽象之上。正如 Percy Liang 在前序讲座中强调的：规模化的关键一环是“有效利用资源”；如果你不了解自己的系统，就永远无法高效利用资源。

在深入硬件之前，主讲人推荐了几份优质学习资源，供有兴趣的读者进一步探索：

- Horace He 关于 GPU 工作原理的博客与讲义；
- GPU Mode 社区，专注于 GPU 内核与底层库的讨论；
- 捷克 (Štěpán Džstál) 等人撰写的《Programming Massively Parallel Processors》，该书被主讲人称为“非常了不起的教材”，本课程的作业也借鉴了书中的部分练习。

本节课的结构分为三部分：

1. GPU 硬件与编程模型的宽泛入门；
2. 让 GPU 跑得更快的六个核心技巧；
3. 综合案例：FlashAttention。

到课程结束时，我们将具备从零理解 FlashAttention 设计所需的全部概念。

1.2 算力是语言模型扩展的核心驱动力

语言建模领域的核心驱动力之一，是“投入更多算力以训练或推理出更好的模型”。过去几年，研究重点主要放在训练阶段如何堆叠更多算力；而近一年来，推理阶段的算力扩展同样成为焦点。因此，硬件速度、利用率、芯片数量以及并行化效率，共同决定了技术进步的速度。

算力扩展的三种途径

- 更快的单芯片：提升峰值计算性能；
- 更高的利用率：让已有硬件发挥出更大比例的理论峰值；
- 更多的芯片与更好的并行：通过多机多卡横向扩展总吞吐量。

在 20 世纪 90 年代，计算机性能的提升主要依赖 CPU 串行执行速度的增长：晶体管尺寸不断缩小，CPU 主频持续提高。这种规律被称为 **登纳德缩放** (Dennard Scaling)，它描述了晶体管密度增加的同时，功耗密度保持不变的理想情况。然而，登纳德缩放在 21 世纪初已经终结——晶体管数量仍在增长，但进一步缩小不再带来主频提升，因为物理极限导致功耗和散热问题无法忽视。

这意味着，我们不能再依靠“让单条指令执行得更快”来扩展算力。对于大语言模型这样的计算密集型任务，出路只能是 **并行扩展**：不再纵向提升主频，而是横向增加可同时执行指令的计算单元数量。这正是 GPU 与 TPU 等加速器的设计理念。

1.3 从 CPU 串行到 GPU/TPU 并行：后登纳德时代的算力增长

图 1 展示了 NVIDIA GPU 各代际的浮点算力增长。从早期的 K20、M40 到 P100、V100，再到近年的 A100、H100，计算能力呈现出近乎超指数级的跃升。主讲人指出，2017 年 Volta 架构引入的 **张量核心** (Tensor Core) 是关键硬件创新之一；随后结构化稀疏、FP8 等低位宽格式的加入，也持续推高了峰值算力。

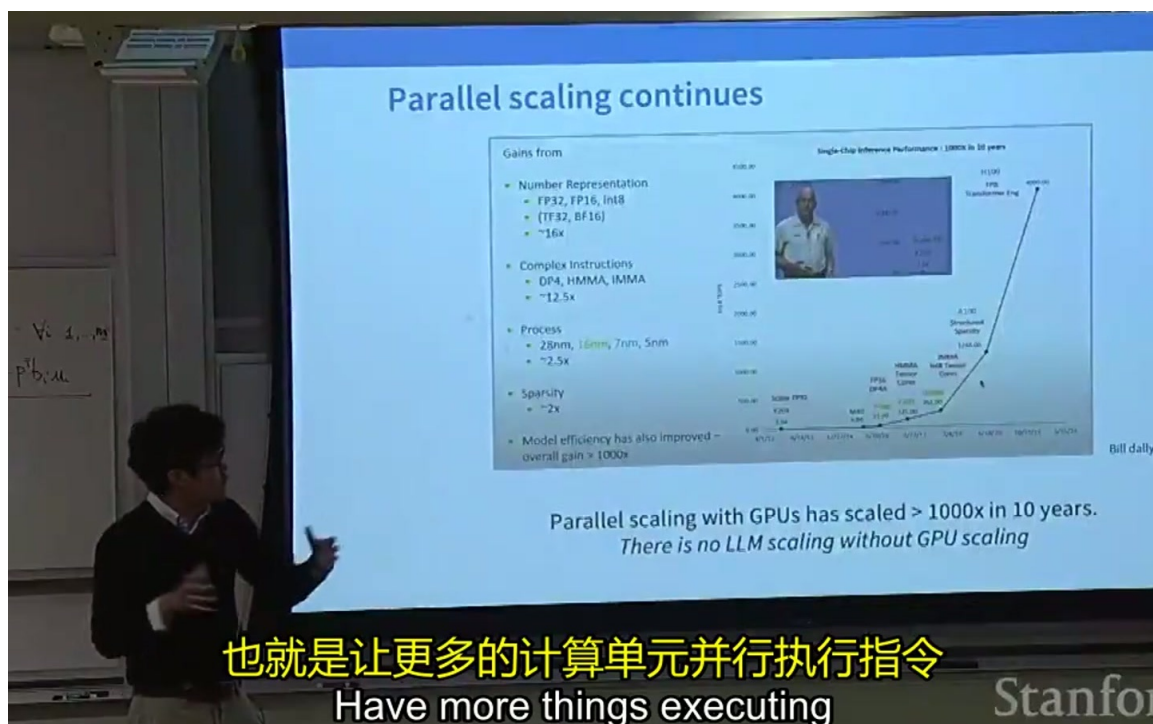


图 1: NVIDIA GPU 单芯片推理性能在十年内增长超过 1000 倍。图中展示了从 K20/M40 到 P100/V100、A100 再到 H100 的 FP32/Tensor Core 算力演进，并总结了数值表示、复杂指令、制程与稀疏性带来的综合增益。¹

术语警示: GPU Tensor Core vs. TPU Tensor Core

GPU 中的“张量核心” (Tensor Core) 指的是 NVIDIA GPU 中专用于矩阵乘累加的硬件单元；而 TPU 中也使用“Tensor Core”一词，却通常指代其矩阵乘法单元 MXU。两者 **同名不同义**，阅读文献时必须根据上下文区分。后文将详细说明。

¹原视频时间戳：00:06:20--00:06:22。

这种从 CPU 串行到 GPU/TPU 并行的转变，可以用一个简洁的公式来概括：在登纳德缩放时代，性能提升主要依赖主频 f ；而在后登纳德时代，性能提升更多依赖并行度 P ：

$$\text{吞吐量} \approx P \times f \times \text{每周期每单元运算数}$$

当 f 难以继续提升时，增大 P （即同时工作的计算单元数量）成为唯一可持续的路径。GPU 正是通过封装成百上千个轻量级计算单元，以 **吞吐量**（throughput）而非 **延迟**（latency）为核心优化目标，实现了这一转型。

图 2 示意了 CPU 主频与晶体管数量随时间的变化关系：晶体管数量持续上升，但主频在 21 世纪初期趋于平台。这一“主频墙”直接催生了现代加速器架构的繁荣。

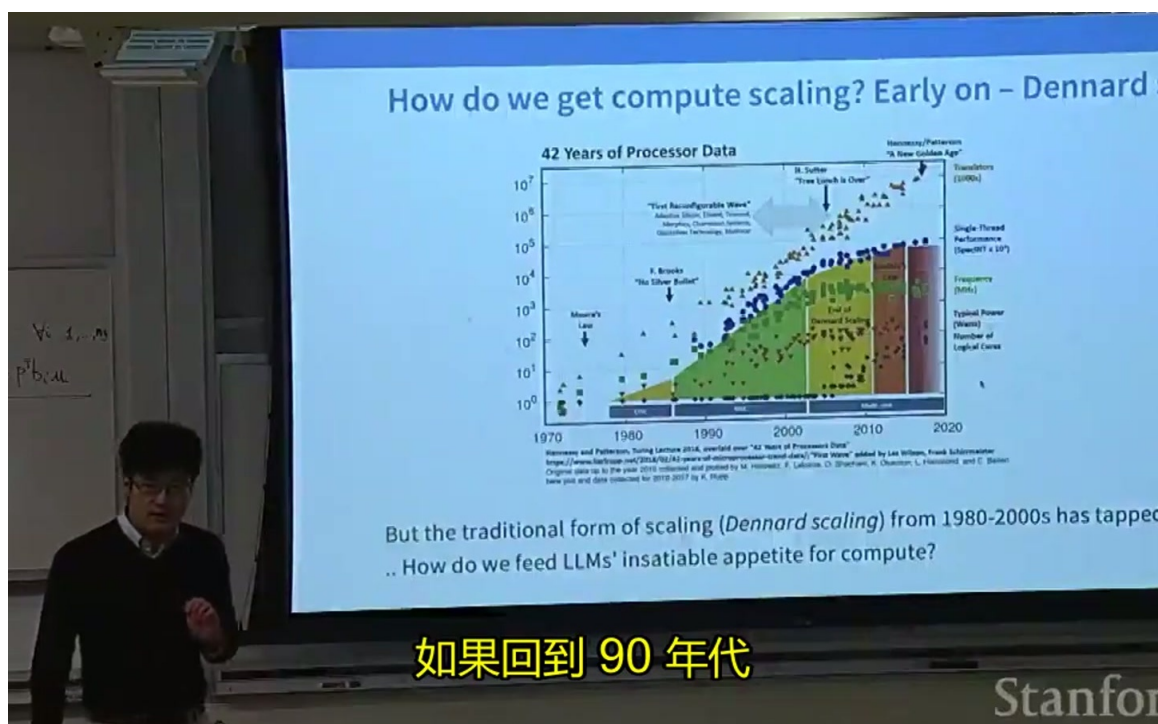


图 2: 处理器 42 年发展数据：登纳德缩放（Dennard Scaling）在 1980–2000 年代推动晶体管数量与主频同步提升，但 21 世纪初后因功耗密度限制而终结，算力增长不得不从提高单核主频转向并行扩展。²

横向扩展 vs. 纵向扩展

- **纵向扩展**（Scale Up）：让单个核心更快，例如提升 CPU 主频；
- **横向扩展**（Scale Out）：增加并行计算单元的数量，例如 GPU 增加 SM、TPU 增加 MXU。

在后登纳德时代，横向扩展是语言模型算力增长的主要来源。

1.4 本章小结

本章作为系统模块的引言，强调了以下几点：

²原视频时间戳：00:05:30--00:05:32。

1. **系统知识是设计高效模型的前提。** 无论是否从事系统方向，理解硬件运行模型都是进行模型架构分析和性能优化的基础。
2. **算力是语言模型扩展的核心驱动力。** 训练与推理的规模化，都依赖于更快、更多、利用率更高的计算资源。
3. **后登纳德时代的算力增长依赖并行。** CPU 主频提升已遇物理瓶颈，GPU/TPU 通过横向增加大量并行计算单元，实现了吞吐量的持续跃升。
4. **GPU 的演进以专用矩阵乘法硬件为分水岭。** 从 Volta 架构引入 Tensor Core 开始，矩阵乘法成为机器学习中被特殊对待的核心操作，也塑造了后续模型架构的设计思路。

在下一章中，我们将从 GPU 的物理结构出发，详细介绍 SM（流式多处理器）、多级存储层次，以及 CUDA 编程模型中的线程、线程块与 Warp，为后续六个优化技巧奠定硬件直觉。

2 GPU 硬件与编程模型

本章将揭开 GPU 的神秘面纱，从芯片物理结构出发，逐步建立 CUDA 编程模型的直觉。理解这些概念是后续所有优化技巧的基石。

2.1 CPU vs. GPU：延迟 vs. 吞吐的设计理念

CPU 和 GPU 代表了两种截然不同的芯片设计哲学。CPU（Central Processing Unit，中央处理器）为快速串行执行而设计，擅长处理复杂的分支逻辑、条件判断和控制流。因此，CPU 上面积很大的部分是控制单元，而算术逻辑单元（ALU）相对较少。它的核心目标是**低延迟**：从拿到指令到执行完毕，时间要尽可能短。

相比之下，GPU（Graphics Processing Unit，图形处理器 / 通用图形处理器）的核心目标是**高吞吐量**。在 GPU 中，单个任务可能需要较长时间才能完成，调度器也会在多个任务之间快速切换；虽然每个任务完成的并不快，但总体上却能获得极高的总吞吐量。这得益于 GPU 内部集成了成百上千个轻量级计算单元，密集地封装在同一块芯片上，并能够并行执行大量指令。

设计理念对比

- CPU：低延迟、强控制流、复杂分支、少量 ALU。
- GPU：高吞吐、弱控制流、简单分支、海量并行 ALU。

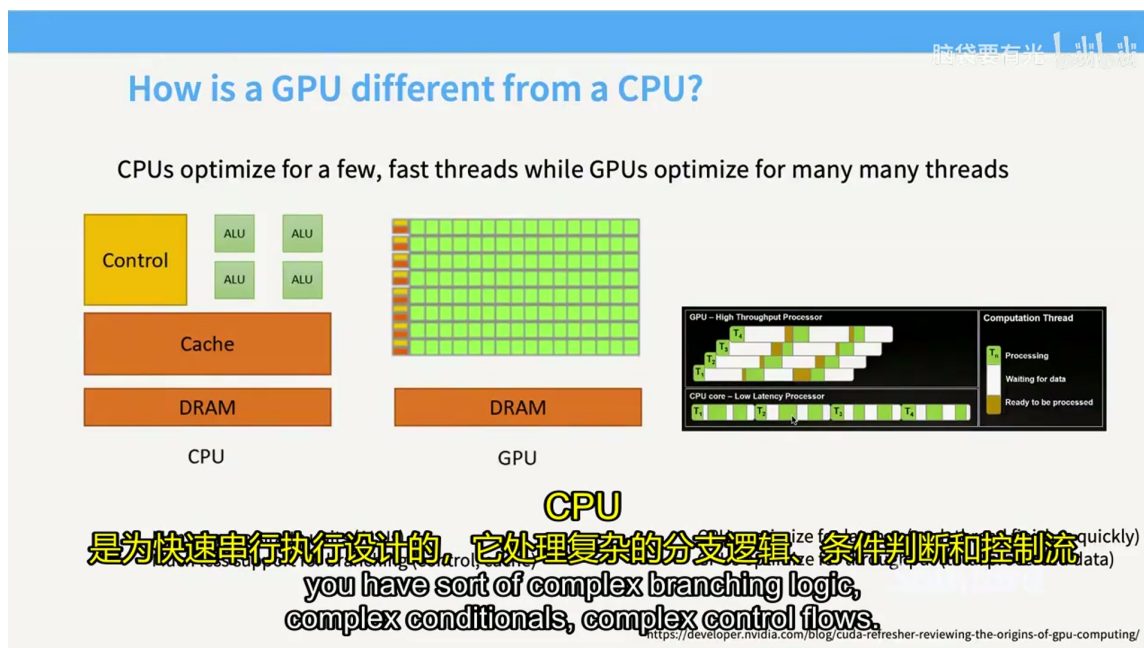


图 3: CPU 与 GPU 的芯片面积/功能单元对比。CPU 将大量面积用于控制逻辑与缓存，以优化少量快速线程的延迟；GPU 则将绝大多数面积用于大量小型计算单元（ALU），以优化整体吞吐量。³

上图展示了 CPU 与 GPU 芯片面积分配的对比。CPU 将大量面积用于缓存和控制逻辑，而 GPU 则将绝大部分面积用于计算单元。这种差异直接决定了我们在 GPU 上编写程序时的思维方式：不再是让一条指令跑得更快，而是让成千上万个线程同时做有用功。

2.2 SM 与物理布局

GPU 的基本独立计算单元称为 SM（Streaming Multiprocessor，流式多处理器）。可以把每个 SM 理解为 GPU 的一个“核心”：它是一个独立的计算单元，内部包含若干子组件（如 CUDA Core、Tensor Core、寄存器与共享内存），并能访问特定类型的内存。

每个 SM 本身又基于并行思想构建：它内部有多个流式处理器，可以同时执行不同线程；但 SM 与 SM 之间是独立的，可以各自执行不同的任务。现代 GPU 通常包含大量 SM，例如 NVIDIA H100 拥有约 132 个 SM，而更早期的架构也有数十至上百个不等。

从芯片平面布局来看，GPU 可以被描述为“SM 阵列 + 共享基础设施”：

- 绿色的小方块是 SM 阵列；
- 中间或四周分布着 L2 缓存；
- L1 缓存 / 共享内存位于每个 SM 内部；
- 芯片外部通过高带宽内存控制器连接 HBM（High Bandwidth Memory，高带宽内存）。

SM 的关键地位

SM 是 GPU 调度和执行的基本单位。一个线程块（block）保证被调度到同一个 SM 上运行，因此块内的线程可以共享该 SM 的共享内存。这一点在后续讲分块（tiling）时至关重要。

³原视频时间戳：00:07:55。

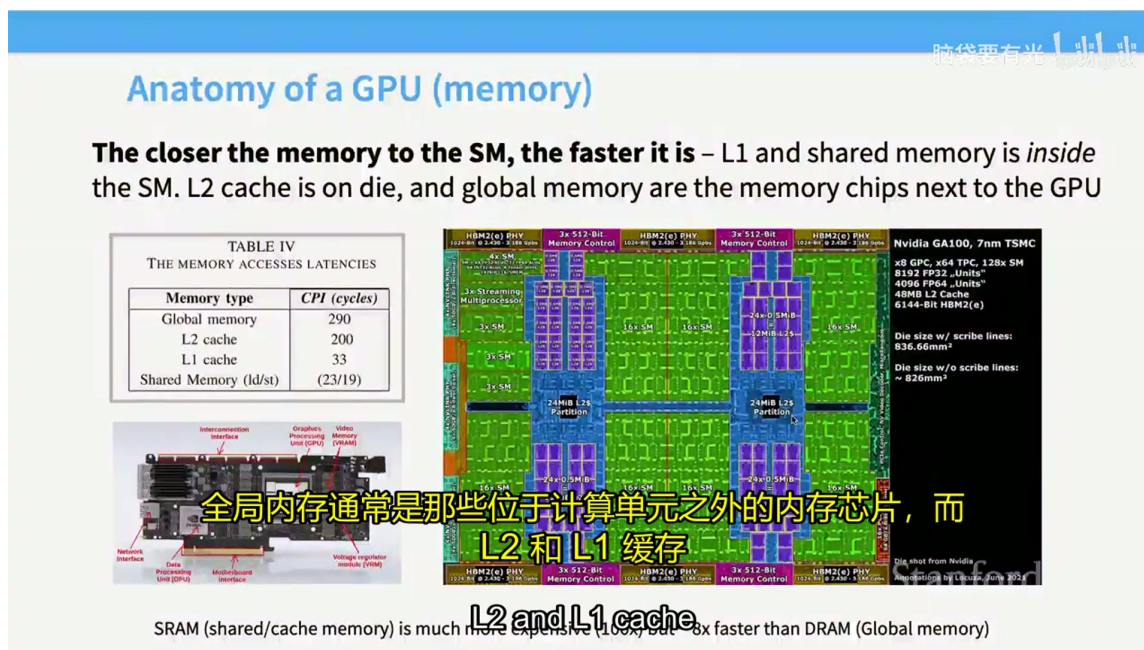


图 4: NVIDIA GA100 GPU 芯片平面布局示意图。绿色区域为 SM 阵列，蓝色区域为 L2 缓存分区，四周为 HBM 控制器，体现了“计算单元围绕高速缓存与内存控制器”展开的物理结构。⁴

2.3 存储层次：寄存器、共享内存/L1、L2、全局内存/HBM

现代硬件和大语言模型的优化，很大程度上由内存决定。GPU 的存储系统呈现鲜明的层次结构，越靠近计算单元的存储越快、容量越小，越远离计算单元的存储越慢、容量越大。

讲座中给出了一张典型延迟对比表（以某代 NVIDIA GPU 为例）：

- **寄存器 (Register)**：最贴近核心，速度最快，容量最小，由编译器自动分配。
- **L1 缓存 / 共享内存 (Shared Memory / SRAM)**：位于 SM 内部，延迟约 20–30 个时钟周期，容量较小，但可由程序员显式管理。
- **L2 缓存**：位于芯片上，被所有 SM 共享，延迟显著高于 L1。
- **全局内存 / HBM**：位于芯片外部，容量大（如 H200 可达 144 GB），但延迟比 L1 高约一个数量级。

⁴原视频时间戳：00:11:50。

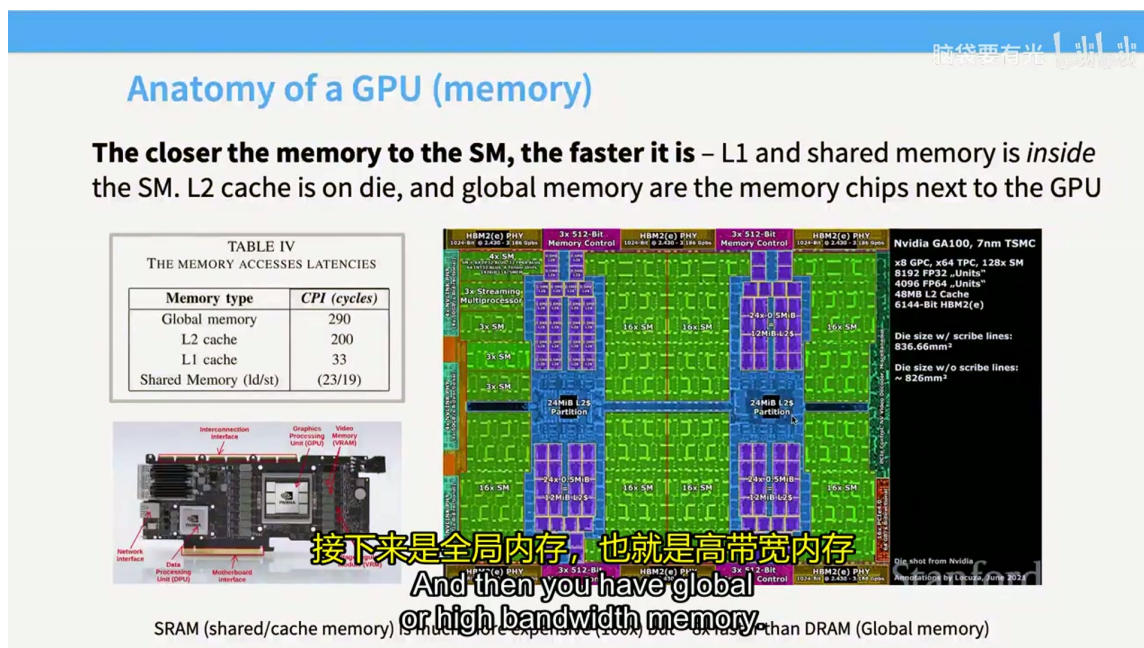


图 5: GPU 存储层次与典型访问延迟。越靠近 SM 的存储越快：共享内存/ L1 约 20–30 个周期，L2 约 200 个周期，全局内存（HBM）约 290 个周期。SRAM 虽昂贵但速度显著高于 DRAM。⁵

从物理位置上看，全局内存通常是计算单元之外的独立内存芯片，而 L2 与 L1 缓存则物理上存在于芯片内部。正因为 L1/共享内存更靠近计算单元，所以延迟要低得多。这也解释了为什么高性能 GPU 程序总是尽可能把数据留在共享内存，而不是频繁访问全局内存。

不要幻想全 SRAM 芯片

有人可能会问：既然共享内存这么快，为什么不把整颗芯片都做成 SRAM？答案是成本和功耗会高出数百倍。因此，实用的加速器必须依赖层次化存储，并在算法层面高效利用它。

另外需要注意，**共享内存与 L1 缓存**的运作方式不同：缓存只是自动保存最近访问的数据，程序员无法直接控制；而共享内存是可编程的，开发者可以显式地把数据放入其中，供线程块内的多个线程复用。讲座中反复强调：一旦访问共享内存之外的数据，速度就会变慢。因此，将线程块分组以减少全局内存读取次数，是贯穿本讲座的核心策略。

2.4 CUDA 执行模型：线程、线程块、Warp 与 SIMD

CUDA 的编程模型中有三个至关重要的抽象层级：线程（thread）、线程块（block / thread block）、线程束（warp）。理解它们之间的关系，是写出高效 GPU 代码的前提。

2.4.1 线程（Thread）

线程是 GPU 中最轻量的并行执行单元，负责完成特定任务。GPU 中的线程遵循 SIMT（Single Instruction, Multiple Threads，单指令多线程）模型：同一个 warp 内的所有线程执行完全相同的指令，但每个线程可以操作不同的数据。这种设计在可编程性和效率之间做了权衡——如果每个线程都能做完全不同的事情，硬件调度将变得极其复杂；而强制相同指令流，则能用简单的控制逻辑换取极高的并行效率。

⁵ 原视频时间戳：00:11:00。

2.4.2 线程块 (Block / Thread Block)

一个线程块是一组保证运行在同一个 SM 上的线程。正因为它们共享同一个 SM，所以块内线程可以访问同一块**共享内存**。这在分块矩阵乘法等场景中非常关键：我们可以先把数据从全局内存加载到共享内存，然后让块内线程反复复用，从而大幅减少昂贵的全局内存访问。

2.4.3 线程束 (Warp)

Warp 是 GPU 内部的调度单元。在 NVIDIA GPU 中，一个 warp 包含 32 个连续编号的线程，它们以 SIMD (Single Instruction, Multiple Data, 单指令多数据) 方式同时执行。调度器每次决定“下一个执行哪个 warp”，而不是逐个线程调度，从而大大降低了调度开销。

这三个层级的包含关系可以概括为：

线程 (Thread) → 线程束 (Warp, 32 个线程) → 线程块 (Block, 若干 warp) → SM (若干 block)

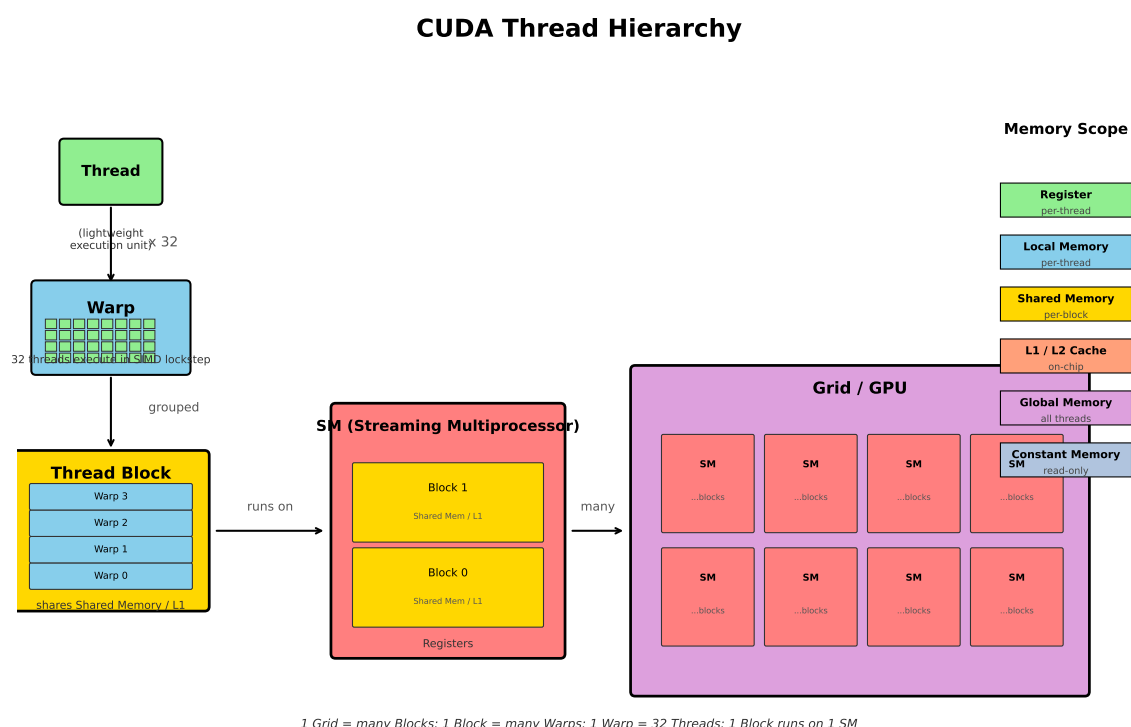


图 6: CUDA 执行模型的层级关系：Thread（线程）→ Warp（线程束，32 线程）→ Block（线程块）→ SM（流式多处理器）→ Grid（整个 GPU）。右侧标注了各级存储的作用域。根据讲座内容重绘。⁶

2.4.4 CUDA 内存模型

在 CUDA 的内存模型中，设备代码可以访问多种存储空间：

- **寄存器 (Register)**：速度最快、最贴近核心，可存放数组起始地址、中间结果等。

⁶原视频时间戳：00:15:20。

- **本地内存 (Local memory)**: 限于单个线程的私有数据, 物理上通常位于全局内存。
- **共享内存 (Shared memory)**: 在线程块内共享, 适合存放多个线程复用的数据。
- **全局内存 (Global memory / HBM)**: 远端大容量内存, 访问延迟高。
- **常量内存 (Constant memory)**: 只读缓存, 实际编程中较少显式使用。
- **主机内存 (Host memory)**: CPU 系统内存, 用于处理超出 GPU 显存容量的数据。

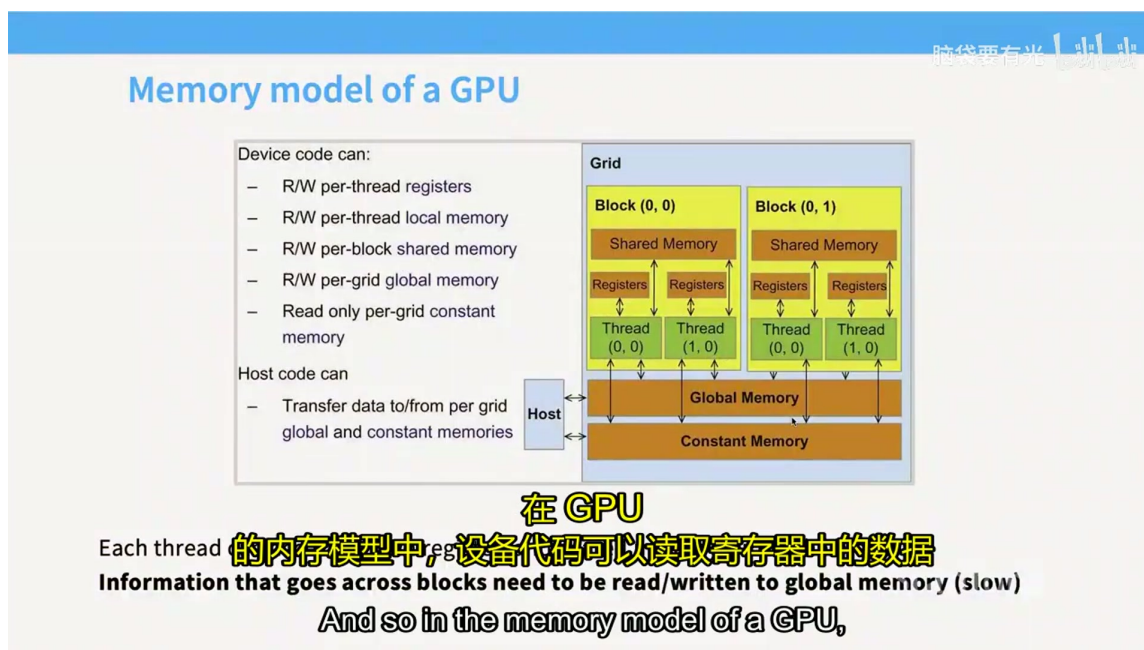


图 7: CUDA 内存模型示意图。设备代码可访问 per-thread 的寄存器/本地内存、per-block 的共享内存、per-grid 的全局内存与只读常量内存; 主机代码负责与全局内存和常量内存进行数据传输。⁷

CUDA 编程的核心直觉

将线程块分组, 并尽量把数据复用限制在共享内存之内, 是减少全局内存访问、提升 GPU 程序性能的核心策略。后续章节介绍的算子融合、合并访问、分块等技巧, 本质上都围绕这一直觉展开。

2.5 本章小结

本章从硬件视角建立了 GPU 的基本直觉, 核心要点如下:

1. **设计理念不同**: CPU 追求低延迟和复杂控制流, GPU 追求高吞吐量和海量并行。
2. **SM 是基本计算单元**: GPU 由大量流式多处理器 (SM) 组成, 每个 SM 内部又有众多计算核心和存储资源。
3. **存储层次决定性能**: 寄存器最快、共享内存/L1 次之、L2 再次、全局内存/HBM 最慢。高性能程序必须尽可能利用靠近计算单元的存储。

⁷ 原视频时间戳: 00:16:05。

4. **CUDA 执行模型三层级**：线程 (thread) → 线程束 (warp, 32 线程) → 线程块 (block) → SM。线程块保证运行在同一 SM 上，因此可共享共享内存。
5. **编程核心策略**：减少全局内存访问、在线程块内复用共享内存，是将硬件直觉转化为高效实现的第一步。

掌握了这些基础后，下一章将自然过渡到 TPU：我们会看到，GPU 与 TPU 在宏观上呈现出“趋同演化”——二者都拥有矩阵乘法单元、层次化存储和海量并行，只是模块规模和灵活性有所不同。

3 TPU 简介与 GPU/TPU 对比

3.1 为什么需要了解 TPU

本章大部分内容围绕 GPU 展开，因为日常模型训练与推理中，GPU 是最常用的加速器。然而，主讲人特意用两页幻灯片介绍 TPU (Tensor Processing Unit, 张量处理器)，原因有二：其一，TPU 近年来越来越流行，尤其在 Google 的模型训练与推理基础设施中占据核心地位；其二，TPU 与 GPU 的对比恰似生物学中的**趋同演化** (convergent evolution) ——两者从不同的设计起点出发，最终却走向了极为相似的宏观架构。理解 TPU，有助于我们抓住加速器设计的通用原则，而不是把 GPU 的特殊实现误当成唯一真理。

为什么要对比 GPU 与 TPU

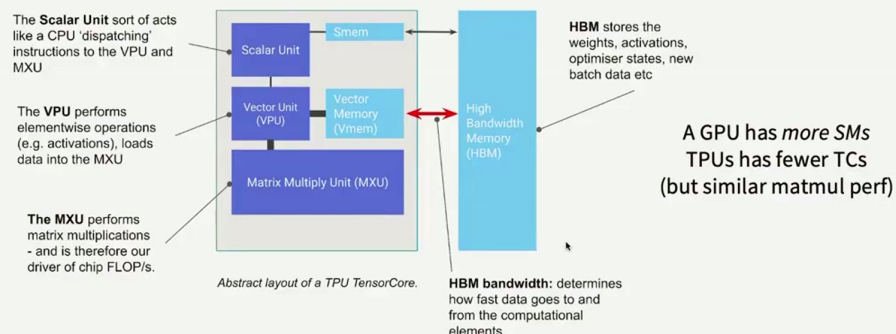
- **工程现实**：越来越多的大规模训练集群使用 TPU，了解其编程模型有助于迁移与调试。
- **设计直觉**：GPU 与 TPU 在存储层次、矩阵乘法单元、海量并行等核心概念上高度相通，掌握一端即可快速理解另一端。
- **避免术语混淆**：两者都使用“张量核心” (Tensor Core) 一词，但所指硬件单元截然不同，必须根据上下文区分。

3.2 TPU 的宏观结构：控制器、向量单元、MXU/脉动阵列

从宏观上看，TPU 的组成非常简洁：一个专用的矩阵乘法电路、一个能执行并行向量运算的组件，以及一个控制系统。它的内存架构与 GPU 几乎一致：远端是容量大但速度慢的 HBM (High Bandwidth Memory, 高带宽内存)，近端是速度更快的本地 SRAM/共享内存。图 8 展示了 TPU 的整体结构。

Side thread – What about TPUs?

GPUs, TPUs, and many other accelerators are at a high level, similar



Core structure – lightweight control, fast (big) matmul unit, fast memory.

Differences - how the accelerators are networked (in the parallelism lecture)

- no warps (just blocks – tradeoffs in matmul vs non-matmul)

图 8: TPU Tensor Core 的抽象架构示意图。图中展示了标量单元 (Scalar Unit) 负责分发指令、向量单元 (VPU) 执行逐元素操作、矩阵乘法单元 (MXU) 执行矩阵乘累加, 以及 HBM 与片上 Smem/VMem 的存储层次。该架构在概念上与 GPU 非常相似, 核心差异体现在各模块的规模与灵活性上。⁸

TPU 中执行矩阵乘法的核心单元称为 **MXU** (Matrix Multiply Unit, 矩阵乘法单元)。MXU 基于**脉动阵列** (systolic array) 构建: 数据像流水一样在阵列中流动, 每个处理单元在数据经过时完成一次乘累加操作, 最终汇总得到矩阵乘积。这种结构的优点在于, 一旦数据流被启动, 计算与数据移动可以高度重叠, 从而在单位面积内获得极高的矩阵乘法吞吐量。

脉动阵列的核心思想

脉动阵列的名字来源于“心跳”: 数据以规律的节奏从一个处理单元流向下一个处理单元, 每个单元在每个时钟周期都做同样的乘累加。与 GPU Tensor Core 相比, TPU 的 MXU 数量更少、单个规模更大, 因此更适合规则的大型矩阵乘法, 而在不规则或小型运算上灵活性较低。

图 9 示意了脉动阵列的数据流: 左侧矩阵 A 的元素从上向下、右侧矩阵 B 的元素从左向右进入阵列, 经过若干周期后, 结果矩阵 C 的元素从右下角逐次流出。整个过程中, 每个处理单元只与相邻单元通信, 避免了频繁访问全局内存。

⁸ 原视频时间戳: 19:00。

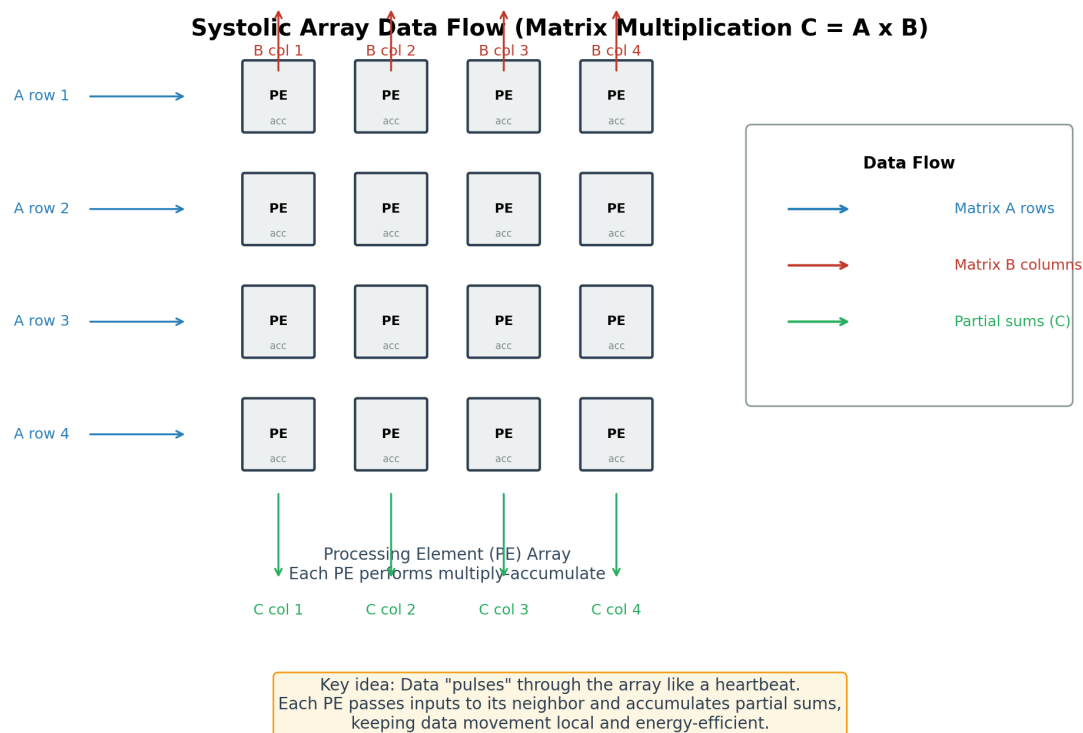


图 9: 脉动阵列 (Systolic Array) 数据流示意图。矩阵 A 的行从左向右、矩阵 B 的列从上向下流入处理单元 (PE) 阵列, 每个 PE 执行乘累加并将输入传递给相邻单元; 部分和最终从阵列底部流出。数据像心跳一样在阵列中脉动, 保持局部移动, 从而实现高能效的矩阵乘法。⁹

3.3 GPU 与 TPU 的“趋同演化”与关键差异

主讲人指出, GPU 与 TPU 在概念层面几乎可以一一映射:

- GPU 的 SM (流式多处理器) 对应 TPU 的独立计算单元;
- GPU 的 Tensor Core (张量核心) 对应 TPU 的 MXU (矩阵乘法单元);
- GPU 的共享内存/SRAM 对应 TPU 的本地高速内存;
- GPU 的 HBM/全局内存对应 TPU 的 HBM。

这种对应关系如此之精确, 以至于捷克等人撰写的《Programming Massively Parallel Processors》在讲解 GPU 时, 专门用表格把 GPU 概念映射到 TPU 上。然而, 两者在**规模与灵活性**上存在显著差异。

术语警示: GPU 的“张量核心”与 TPU 的“张量核心”同名不同义

NVIDIA GPU 把 SM 中的矩阵乘累加单元称为 **Tensor Core** (张量核心); Google TPU 也把 MXU 称为 **Tensor Core**。前者数量多、规模小、灵活性强; 后者数量少、规模大、专用于大型矩阵乘法。阅读文献时必须根据上下文判断具体所指。

⁹根据讲座内容重绘。

图 10 给出了 GPU 与 TPU 的关键参数对比。以 H100 GPU 为例，它拥有约 132 个 SM，每个 SM 配备 4 个 Tensor Core，总计约 528 个矩阵乘法单元；而 TPU 仅有 2 个独立计算单元和 8 个 MXU。TPU 的 MXU 拒绝接受小于 64 的矩阵维度输入，这意味着它只能高效执行大规模矩阵乘法；相比之下，GPU 的大量小型 Tensor Core 可以灵活地处理各种尺寸的运算，并通过编程实现更丰富的操作类型。

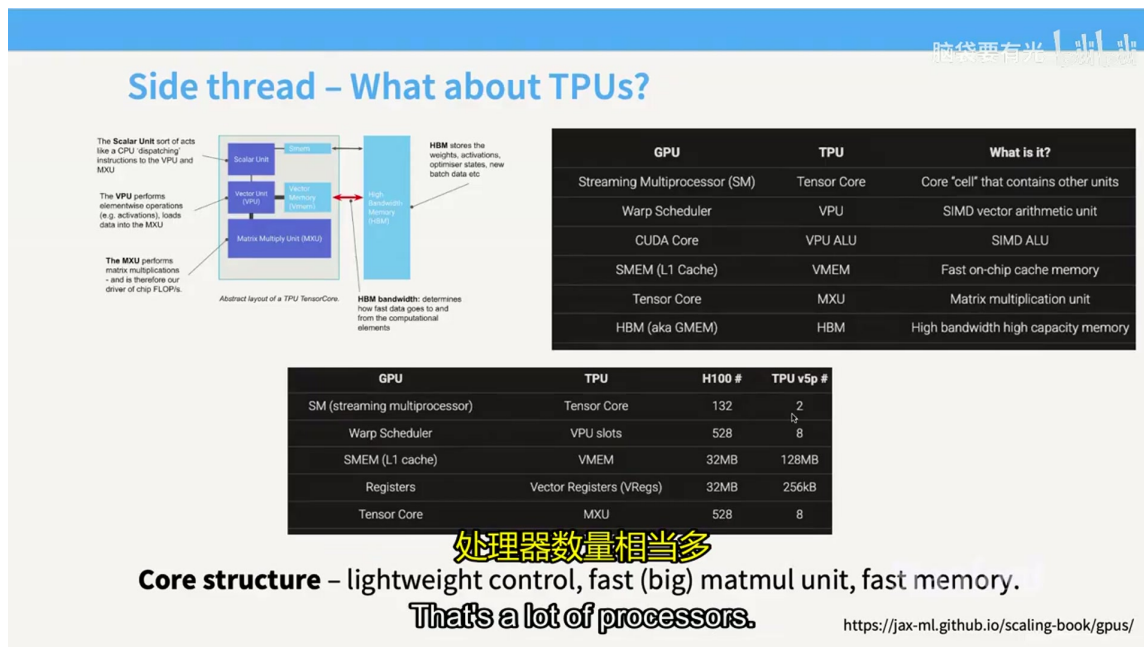


图 10: GPU 与 TPU 关键参数对比表。上图给出了 GPU 组件与 TPU 组件的术语对照；下图以 H100 与 TPU v5p 为例，展示了两者在 SM/Tensor Core 数量、调度槽位、片上缓存/寄存器容量以及矩阵乘法单元数量上的显著差异：GPU 拥有 132 个 SM 和 528 个 Tensor Core，而 TPU v5p 仅有 2 个 Tensor Core 和 8 个 MXU，但每个 MXU 规模更大。¹⁰

这种差异反映了两条不同的设计哲学：

1. **GPU：用大量小型单元换取灵活性。**更多的 SM 和 Tensor Core 意味着更容易通过增加单元数量来扩展吞吐量，同时也支持更广泛的并行模式和非矩阵运算。
2. **TPU：用少量大型单元换取专用效率。**更大的 MXU 在规则的大型矩阵乘法上能达到极高的面积效率与能效比，但对工作负载的形状和规模有更强假设。

GPU 成功的关键：矩阵乘法的可扩展性

GPU 之所以在深度学习领域大获成功，一个重要原因是它的架构让矩阵乘法可以**轻松扩展**：只要显存带宽跟得上，不断增加 SM 就能持续提升吞吐量。自 Volta 架构引入专用 Tensor Core 以来，矩阵乘法成为机器学习中被特殊对待的核心操作。如今，在可并行化但非矩阵乘法的操作与矩阵乘法操作之间，吞吐量可能存在十倍以上的差距。这也是为什么几乎所有未来能够扩展算力的机器学习架构，都会包含矩阵乘法。

3.4 本章小结

本章从 TPU 的宏观结构出发，对比了 GPU 与 TPU 的设计理念：

¹⁰原视频时间戳：21:00。

1. **TPU 与 GPU 在架构上高度相似**，都拥有矩阵乘法单元、层次化存储和海量并行，概念几乎可以一一对应。
2. **TPU 的核心是 MXU**，基于脉动阵列实现大规模矩阵乘法；其控制器更轻量，专用性更强。
3. **GPU 与 TPU 的主要差异在于单元规模与灵活性**：GPU 拥有大量小型 Tensor Core，灵活性高、易于扩展；TPU 拥有少量大型 MXU，专用性强、对大型规则矩阵乘法效率极高。
4. **“张量核心”一词在两种硬件中同名不同义**，阅读文献和讨论时必须根据上下文区分。
5. **趋同演化的底层原因**：高效分配内存的方式有限，快速做矩阵乘法的方法也有限，GPU 与 TPU 最终殊途同归。

理解这些硬件直觉后，下一章将把它们转化为具体的优化技巧。无论面对的是 GPU 还是 TPU，核心目标始终如一：**减少不必要的内存移动，让计算单元尽可能饱和地工作。**

4 让 GPU 跑得更快的六个技巧

第三章指出，GPU 与 TPU 虽然设计取向不同，却呈现出惊人的“趋同演化”：二者都依赖海量并行、层次化存储和专用矩阵乘法单元。本章将把这些硬件直觉归纳为六个具体技巧。主讲人在本节开头展示了一张“谜题”曲线——矩阵乘法吞吐量随矩阵维度上下波动——并承诺在学完这六个技巧后，我们就能读懂这张图背后的全部原因。

4.1 屋顶线模型与计算强度

在深入技巧之前，需要先建立一个判断性能瓶颈的思维框架：**屋顶线模型**（Roofline Model）。

现代 GPU 内部各组件的发展速度并不一致。如图 11 所示，计算吞吐量的增长（灰色）远快于内存带宽（绿色）和互联带宽（蓝色）。这意味着：越新一代的 GPU，计算与内存之间的差距越大。因此，今天绝大多数优化都不是“让计算更快”，而是“让内存不要成为瓶颈”。

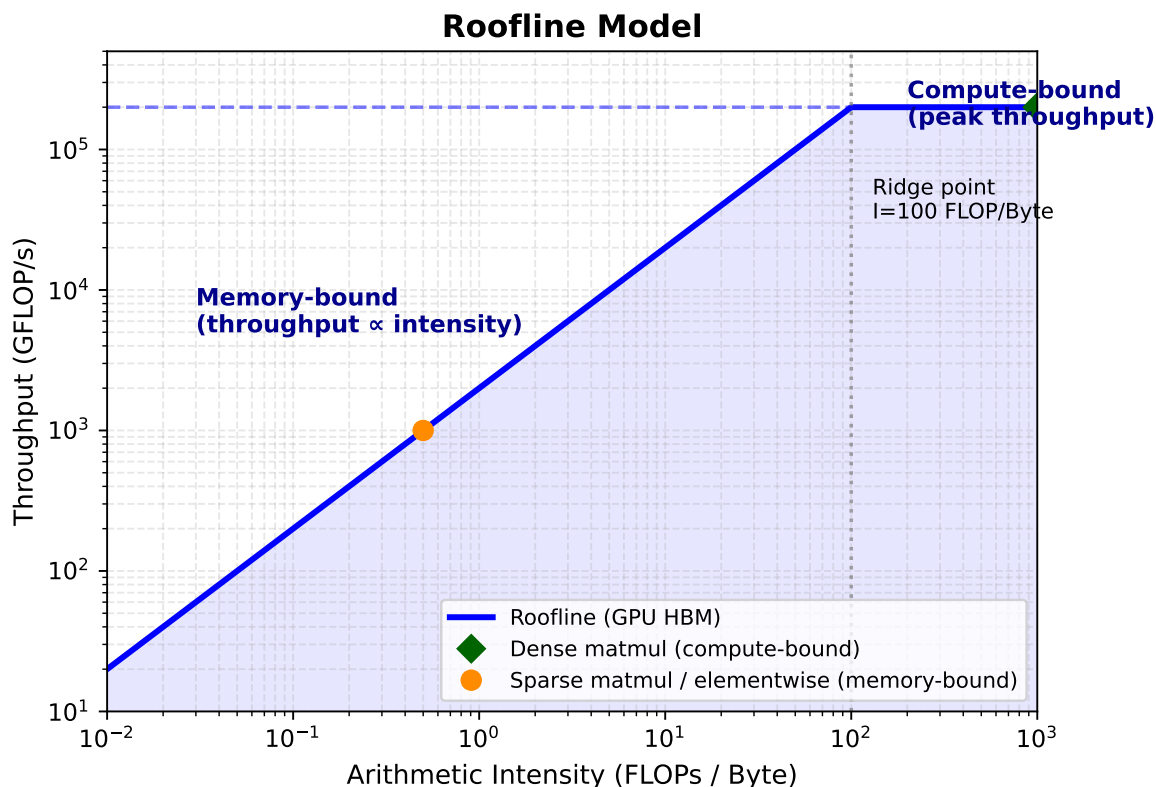


图 11: 屋顶线模型示意图: 横轴为计算强度 (FLOPs/Byte), 纵轴为吞吐量; 左侧对角线区域受内存带宽限制, 右侧平坦区域受计算峰值限制。¹¹

屋顶线模型用**计算强度** (Arithmetic Intensity) 作为横轴, 其含义是:

$$\text{计算强度} = \frac{\text{浮点运算次数}}{\text{字节传输量}}$$

纵轴则是实际可达吞吐量。模型呈现两段:

- **左侧对角线区**: 计算强度低, 性能被内存带宽封顶。此时无论算力多强, 每读一个字节对应的工作量太少, GPU 只能等待数据。
- **右侧平坦区**: 计算强度足够高, 矩阵乘法单元已满载, 性能达到峰值算力。

关键直觉

写高效 GPU 代码的核心目标, 就是让任务尽可能落在屋顶线模型的**平坦区**——也就是提高每次内存读取所对应的计算量。本章的六个技巧中, 除了第一个与控制发散相关, 其余五个本质上都在减少内存移动或提高计算强度。

4.2 技巧 1: 避免控制发散

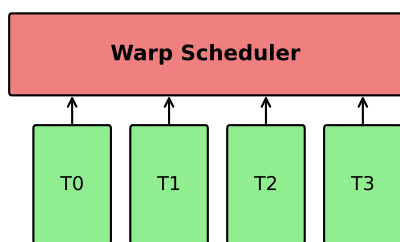
GPU 采用 SIMT (单指令多线程) 执行模型: 一个 Warp (32 个线程) 在同一时刻执行同一条指令。这带来了一个与 CPU 截然不同的现象——**控制发散** (Branch Divergence)。

如果在 CUDA 代码中写了一条 `if/else`, 而同一个 Warp 内的线程走了不同分支, GPU 并不会像 CPU 那样只执行其中一个分支。相反, 它会**串行执行两个分支**: 先执行 `if` 分支, 不满足条件

¹¹ 根据讲座内容重绘。

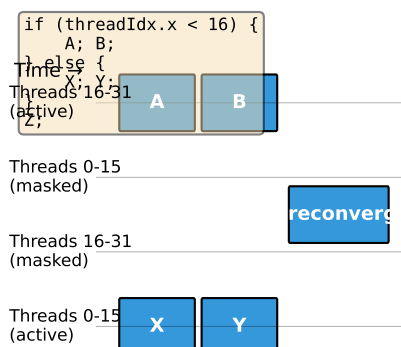
的线程被屏蔽而空转；再执行 `else` 分支，刚刚活跃的线程现在空转。如图 12 所示，一个 Warp 被拆成两部分，有效利用率大幅下降。

SIMT: Single Instruction, Multiple Threads



All threads in a warp execute the same instruction together

Control Divergence in a Warp



Divergent branches serialize execution, reducing utilization

图 12: Warp 内控制发散示意图：同一线程束中的线程因条件分支被拆成两组，分别执行两个分支，导致部分线程空转。¹²

正因如此，高性能 GPU 内核往往避免使用 `if`，而改用**掩码乘法**：用 0 或 1 的掩码与张量相乘，一次性完成“条件选择”。虽然看起来多算了几个乘法，但避免了分支带来的 Warp 空转，总体更快。

实践提示

在阅读 `relu`、`dropout` 等底层 CUDA 实现时，常见模式是 `mask * x` 而非 `if (condition) x = 0;`。这种写法正是为了避免控制发散。

4.3 技巧 2：低精度计算与量化

主讲人把低精度训练称为“英伟达投入最多的硬件方向”，并认为它是 GPU 算力爆炸式增长的重要来源之一。思路非常直接：把每个数从 32 位降到 16 位、8 位甚至 4 位，需要搬运的数据量就减半再减半，从而缓解内存带宽瓶颈。

4.3.1 从 FP32 到 BF16

- **FP32**: 1 位符号 + 8 位指数 + 23 位尾数，共 32 位。这是传统单精度训练的标准格式。
- **BF16** (Brain Floating Point): 1 位符号 + 8 位指数 + 7 位尾数，共 16 位。它保留了与 FP32 相同的 8 位指数，因此动态范围接近 FP32，只是精度降低。对深度学习训练而言，这通常比 FP16 (1/5/10) 更稳定。

仅将 FP32 换成 BF16，每次内存访问的数据量就减半。主讲人举了一个极简例子：对单个浮点数做一次运算，FP32 需要读 4 字节、写 4 字节，共 8 字节/次运算；BF16 只需 4 字节/次运算。这相当于把算术强度翻了一倍。

¹²根据讲座内容重绘。

4.3.2 FP8 与块缩放格式

进一步压缩到 8 位时，问题变得复杂：没有一种“通用 FP8”能满足所有场景。于是出现了多种分配方案，例如 **E4M3**（4 位指数、3 位尾数）和 **E5M2**（5 位指数、2 位尾数），分别强调精度或动态范围。

更前沿的是 **MXFP8**（Microscaling FP8）。它的核心观察是：一个矩阵内部的数值幅度可能差异很大，用单个全局缩放因子会损失太多信息。因此 MXFP8 把矩阵分成小块，每 32 个元素共享一个缩放因子。这些缩放因子本身也是 8 位（E8M0，只有指数），表示 2 的幂。

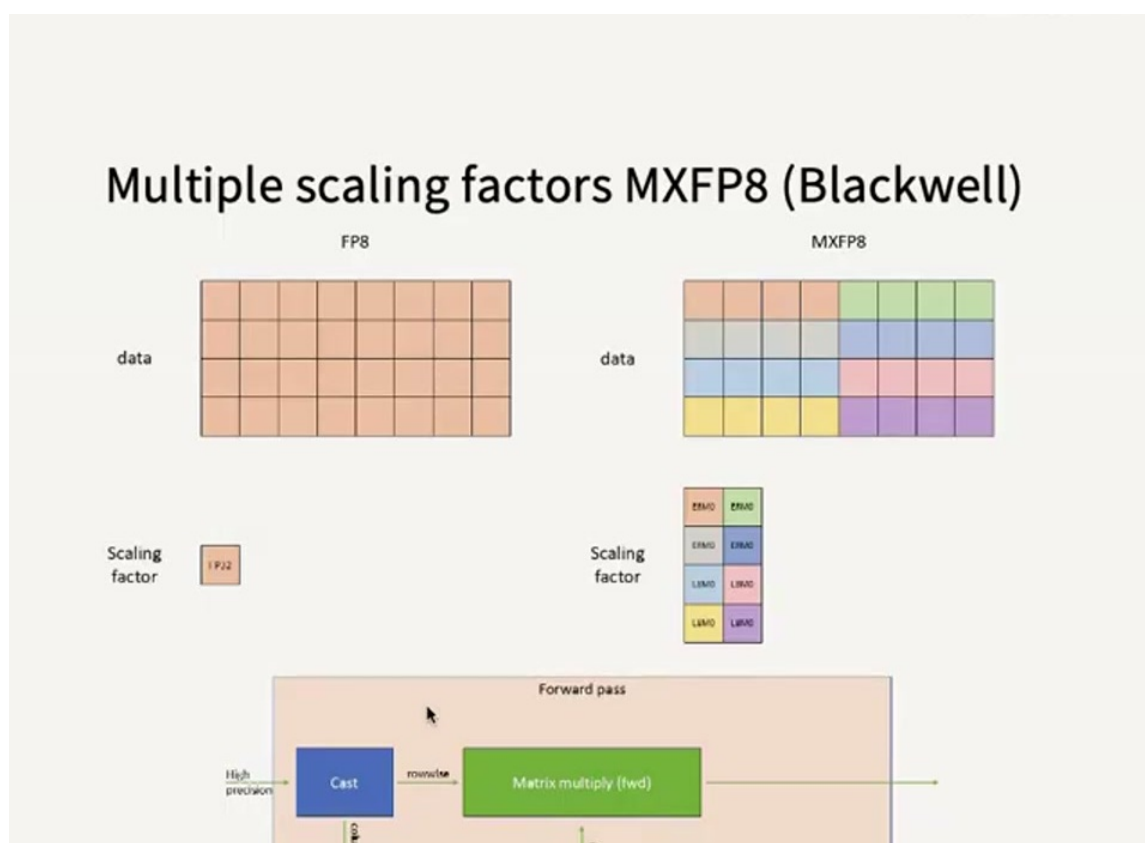


图 13: MXFP8 块缩放示意图：矩阵被划分为若干子块，每 32 个元素共享一个 E8M0 缩放因子；转置后缩放模式改变，需要维护原矩阵与转置矩阵两份量化副本。¹³

MXFP8 带来了一个非常“疯狂”的实现细节：矩阵转置会改变缩放因子的分组方式。如图 13 所示，原矩阵按行方向每 32 个元素一组，转置后变成了列方向。为了在做矩阵乘法时两个方向都能直接读取，系统会同时保存原矩阵和转置矩阵的两份量化副本。这增加了存储开销，但换来了硬件上的高效矩阵乘法。

4.3.3 MXFP4

最新一代格式已经把每个元素压到 4 位（**MXFP4**）。每 16 个元素共享一个 E8M0 缩放因子，可表示的取值范围只有 6 个离散值。主讲人坦言，目前尚未有人成功用 FP4 训练出真正的大模型，但“这一天很快就会到来”。

¹³原视频时间戳：00:39:00。

低精度训练的代价

低精度并不是免费的午餐：

1. 量化和反量化本身需要额外计算；
2. 缩放因子需要遍历数据或根据历史统计动态确定；
3. 某些层对精度敏感，不能盲目量化。以输出层为例，学生问及为何输出层最难量化时，主讲人解释：输出层直接决定预测分布，是损失函数的“一阶因素”（first-order factor），量化误差会沿梯度放大，导致训练不稳定和损失显著上升；输入层和 softmax、指数运算等也有类似敏感性。
4. MXFP8/MXFP4 需要维护原矩阵与转置矩阵两份副本，增加内存占用。

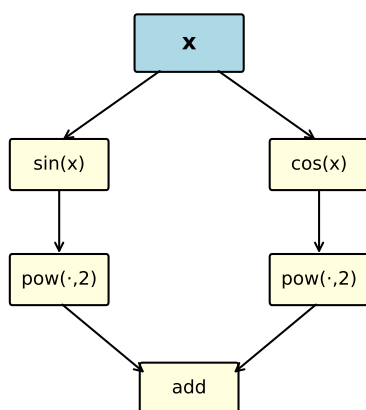
主讲人给出的经验估计是：即便做了所有量化工作，实际收益通常也只有 **20%–30%**，远非简单的“位数减半、速度翻倍”。量化的开销（缩放因子计算、转置副本、精度敏感层保留 FP32/BF16）会吃掉相当一部分理论收益。

4.4 技巧 3：算子融合

算子融合（Operator Fusion）是减少全局内存往返的最直接手段。把 GPU 想象成一座工厂：全局内存是远处的仓库，SM 是车间，内存总线是传送带。如果每个操作都单独从仓库取货、加工、送回，传送带很快就会饱和。

主讲人以 $\sin^2(x) + \cos^2(x)$ 为例。在 PyTorch 的默认计算图中，这会生成四个独立操作：sin、cos、pow（平方）、add。如果不优化，每个操作都要从全局内存读、写一次，共 8 次内存访问（sin 读 1 写 1，cos 读 1 写 1，两个平方各读 1 写 1，最后加法读 2 写 1）。

Before Fusion (5 CUDA kernels)



5 separate kernels → 5 HBM round trips

After Fusion (1 CUDA kernel)

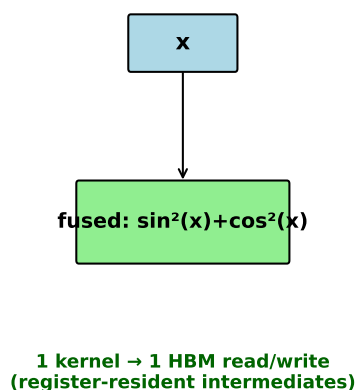


图 14: 算子融合前后对比：未融合时四个操作共需 8 次全局内存访问；融合后只需 1 次读、1 次写，中间结果在 SM 内部完成。¹⁴

¹⁴根据讲座内容重绘。

融合后的内核只做一次全局内存读取，然后在寄存器或共享内存中依次完成 $\sin$ 、平方、 $\cos$ 、平方、相加，最后写回全局内存。如图 14 所示，内存访问从 8 次降到 2 次。

对于简单的逐元素操作链，`torch.compile`、`jax.jit` 等编译器通常能自动完成融合。更复杂的融合（例如把矩阵乘法、偏置、激活、归一化打包在一起）则可能需要手写 CUDA 内核。

4.5 技巧 4：重计算

重计算（Recomputation / Gradient Checkpointing）是“用计算换内存”的经典策略。反向传播时，通常需要保存前向传播的中间激活值。如果模型很深，这些激活值会占用大量 HBM。

主讲人用三层 Sigmoid 网络说明。设输入为 x ，三层激活分别为 s_2, s_1, output ：

- **常规做法**（内存读写共 8 次）：前向时把 x 读入，依次写出 s_2, s_1, output （1 次读 + 3 次写）；反向时读取保存的 s_2, s_1, output 并写回梯度（3 次读 + 1 次写）。
- **重计算做法**（内存读写共 5 次）：前向只保存最终 output （1 次读 + 1 次写）；反向时从 x 重新前向计算 $x \rightarrow s_2 \rightarrow s_1 \rightarrow \text{output}$ ，即时得到所需激活并计算梯度（1 次读 + 3 次写，其中 output 的读可与前向保存合并）。

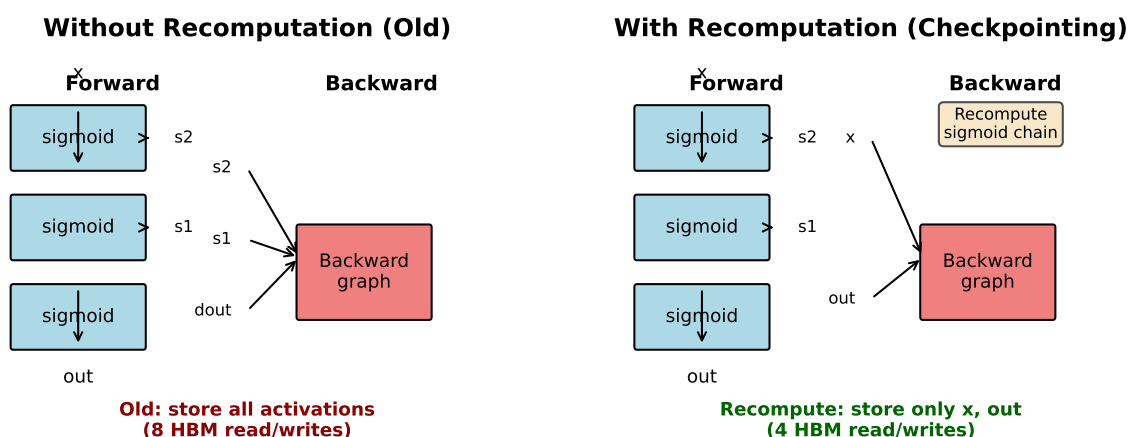


图 15: 重计算前后内存访问对比：常规反向传播需保存并读取所有中间激活；重计算只保存最终输出，反向时实时重新前向计算，减少 HBM 访问。¹⁵

在屋顶线模型的视角下，重计算之所以划算，是因为我们往往处于**计算未满载、内存却吃紧**的区域。多花一点“闲置”的计算，省下昂贵的内存访问，总体反而更快。

4.6 技巧 5：合并内存访问与突发段

DRAM 的物理特性决定了它不是按单个字节读取的，而是按**突发段**（Burst Segment）读取。当控制器访问某个地址时，整个突发段（例如 128 字节）内的数据会一并返回，几乎没有额外开销。因此，让同一 Warp 中的线程连续访问相邻地址——称为**合并访问**（Coalesced Memory Access）——能极大提升有效带宽。

¹⁵根据讲座内容重绘。

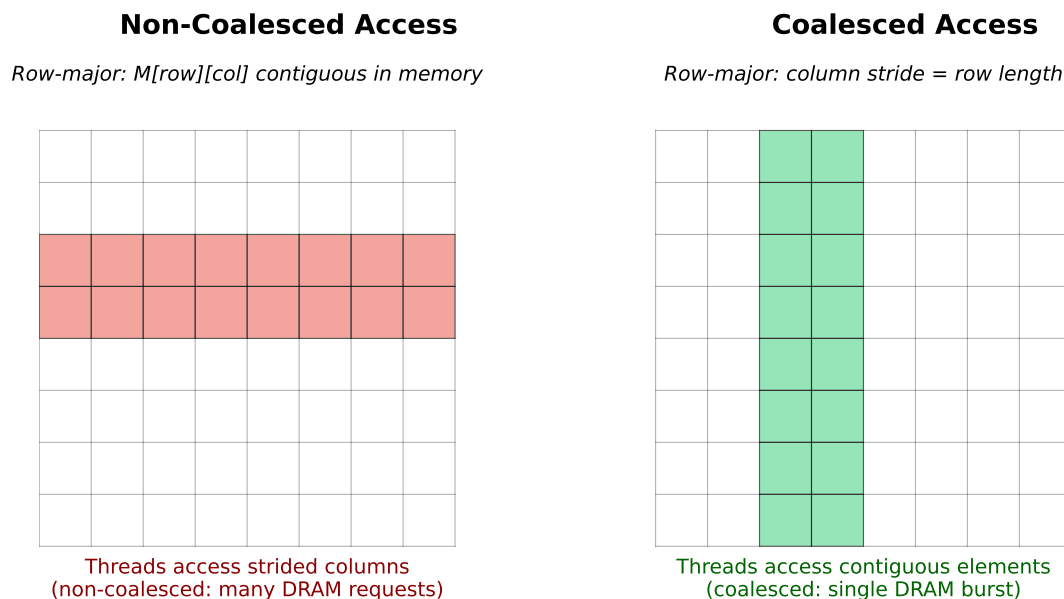


图 16: 行主序矩阵的合并访问 vs. 非合并访问: 左图为非合并的列向读取, 每次访问落在不同突发段; 右图为合并的行向读取, 一次突发传输即可读满连续数据。¹⁶

如图 16 所示, 对于一个按行主序存储的矩阵:

- 如果 Warp 中的线程沿**行方向**连续读取, 它们落在同一个突发段内, 只需一次内存事务;
- 如果沿**列方向**读取, 每个线程访问不同行, 落在不同突发段, 需要多次内存事务, 带宽利用率骤降。

DRAM 的物理直觉

从物理上看, DRAM 的存储单元排列成巨大的网格, 读取某一行需要先激活整行并启动感测放大器, 这个过程耗时很长; 但同一行内连续读取多个列时, 后续列几乎没有额外开销。因此, DRAM 控制器一次会读出整个“突发段”(如 128 字节), 同一 Warp 中的线程若能连续访问相邻地址, 就能充分利用这次突发传输。

当矩阵维度或分块大小与突发段宽度不对齐时, 读取一个分块可能触发两次内存事务(读取两个“参差不齐”的突发段), 有效带宽几乎减半。这就是实践中常常对矩阵进行 **padding** (填充) 的原因: 把行/列长度补到 32、64、128 等对齐边界, 使分块与突发段完美重合。

4.7 技巧 6: 分块与波量化

分块 (Tiling) 被主讲人称为“核心思想”和“对性能影响最大”的技巧。其本质是利用 GPU 的存储层次: 把大矩阵切成适合共享内存 (SRAM) 的小块, 一次性从全局内存搬运到共享内存, 然后在共享内存中反复复用。

4.7.1 分块矩阵乘法

考虑两个矩阵 A 与 B 相乘, 其中 A 为 $M \times K$, B 为 $K \times N$, 得到 C 为 $M \times N$ 。朴素实现中, 每个输入元素要从全局内存读取多次。若将矩阵划分为大小为 $T \times T$ 的分块, 则每个元素从全

¹⁶根据讲座内容重绘。

局内存读取的次数显著下降，进入共享内存后再被反复复用：

$$\text{全局内存读取次数} \propto \frac{N}{T} \quad (\text{而非朴素实现的 } N)$$

当 $T = N$ 时，每个元素只从全局内存读一次，这是最理想的情况（但受共享内存容量限制）。

Tiled Matrix Multiplication: $C = A \times B$

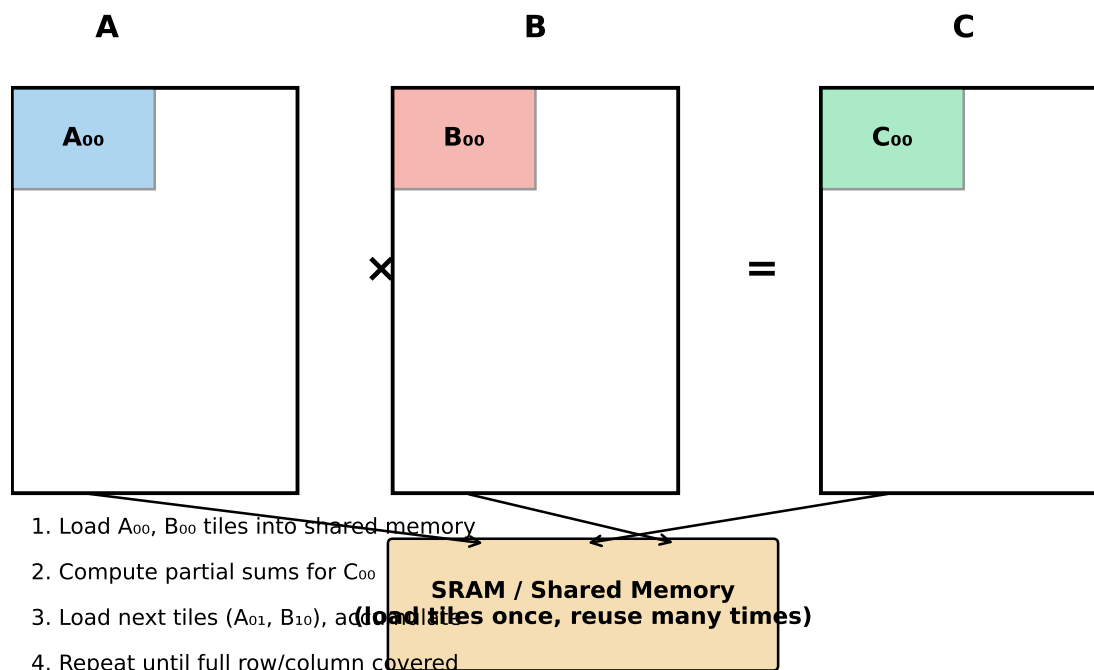


图 17: 分块矩阵乘法示意图：将左矩阵 A 切为 $A_{00}, A_{01}, \dots$ ，右矩阵 B 切为 $B_{00}, B_{10}, \dots$ ，输出矩阵 C 切为 C_{ij} ，每次只把一对子矩阵加载到共享内存中完成局部乘累加。¹⁷

如图 17 所示，算法流程为：

1. 把 A_{00} 和 B_{00} 从全局内存加载到共享内存；
2. 在共享内存中完成子矩阵乘法，累加到输出子块 C_{00} ；
3. 重复加载下一对分块（如 A_{01}, B_{10} ），继续累加；
4. 当整个输出子块计算完毕，再写回全局内存。

4.7.2 波量化

分块虽好，却引入了一个微妙的调度问题，主讲人称之为**波量化**（Wave Quantization）。

以 A100 为例，它有 108 个 SM。假设使用 256×128 的分块大小：

- 当矩阵维度为 1792 时，共产生 98 个分块。108 个 SM 足够一次性全部吞下，所有 SM 都参与计算，利用率接近 100%。
- 当矩阵维度只增加 1 变为 1793 时，分块数跃升到 120 个。108 个 SM 先处理第一波 108 个分块；剩下的 12 个分块需要启动第二波，而第二波运行时其余 96 个 SM 只能空转。

¹⁷根据讲座内容重绘。

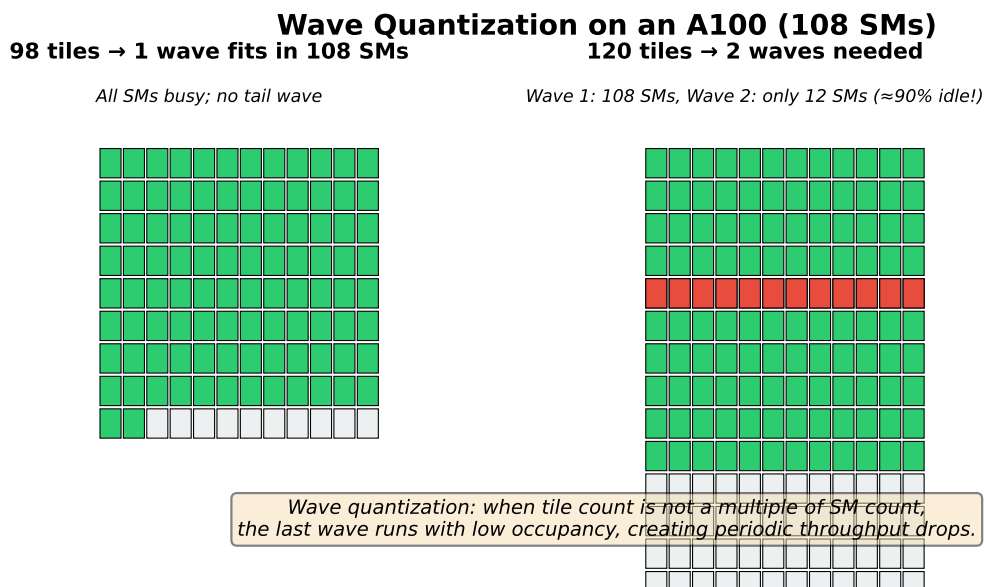


图 18: 波量化示意图：98 个分块可被 108 个 SM 一波处理完毕；120 个分块需要两波，第二波大部分 SM 闲置，导致利用率骤降。¹⁸

如图 18 所示，仅仅把矩阵维度加 1，吞吐量就可能暴跌。这就是本节开头那张“谜题”曲线中周期性低谷的来源之一。

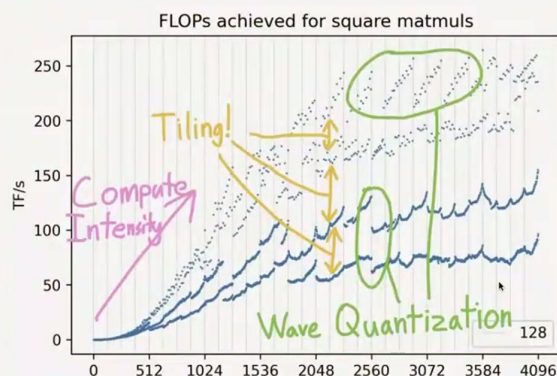
4.7.3 整除性对吞吐量的影响

主讲人还展示了另一张图：把矩阵按“能被多少整除”着色后，曲线变得豁然开朗：

- 蓝色：维度为奇数，吞吐量最差；
- 橙色：能被 2 整除；
- 绿色：能被 8 整除；
- 红色：能被 16 整除；
- 紫色：能被 32 整除，性能最平稳。

¹⁸根据讲座内容重绘。

Matrix mystery



We understand some of this (compute intensity, tiling). Let's take a closer look..

图 19: 矩阵乘法吞吐量随维度变化的“谜题”曲线：按整除性着色后可见，不能被 2 整除的维度性能最差，而能被 16 或 32 整除的维度几乎达到峰值。¹⁹

这并非因为“2 的幂有魔力”，而是因为 16 或 32 恰好匹配突发段宽度，使得分块读取能够合并。实践中，PyTorch 的 `max_autotune` 编译选项会在用户矩阵上尝试多种分块大小，自动寻找最优配置。

分块与填充的实践建议

- 尽量让矩阵维度能被 32、64、128 等数整除；
- 如果 vocab size（如 Andrej Karpathy 在 nanoGPT 速度优化中遇到的 50257）导致不对齐，可以考虑填充到对齐值（如 53040），他报告此举带来了约 **25%** 的吞吐量提升；
- 分块大小本身通常由底层库（cuBLAS、CUTLASS）自动处理，但矩阵尺寸是用户可以直接控制的。

4.8 本章小结

本章围绕“如何让 GPU 跑得更快”展开了六个技巧：

1. **避免控制发散**：减少 Warp 内分支，用掩码乘法替代 `if/else`；
2. **低精度与量化**：从 FP32 到 BF16、FP8、MXFP8、MXFP4，用精度换带宽，但需注意量化层选择、缩放因子维护和转置副本开销；
3. **算子融合**：把多个操作合并为一个内核，减少全局内存往返；
4. **重计算**：反向传播时重新前向计算部分激活，用闲置计算换取内存带宽；
5. **合并内存访问**：利用 DRAM 突发段特性，让 Warp 内线程连续访问相邻地址；

¹⁹原视频时间戳：01:06:20。

6. **分块与波量化**：把大矩阵切到共享内存中复用，同时注意矩阵维度与 SM 数量、突发段宽度的对齐，避免波量化陷阱。

这些技巧的共同点可以用一句话概括：**内存是瓶颈，优化的本质是减少内存移动、提高每次内存访问对应的计算量**。下一章将以 FlashAttention 为综合案例，展示如何把分块、在线 softmax、算子融合和重计算组合在一起，将注意力从典型的内存受限操作转化为高效的融合 CUDA 内核。

5 综合案例：FlashAttention

本章把前面介绍的六项优化技巧串联起来，以 FlashAttention 为例，展示如何从系统视角重新设计一个经典算子。主讲人把 FlashAttention 称为讲座的“压轴部分”——它并非改变了注意力的数学定义，而是通过分块 (tiling)、在线 softmax (online softmax) 与重计算 (recomputation)，将原本内存受限的注意力实现改造成成了一个精心融合的 CUDA 内核。

5.1 问题背景

标准自注意力 (self-attention) 的计算流程可以概括为三步：

1. 计算相似度分数： $S = QK^T$ ；
2. 对分数做 softmax： $P = \text{softmax}(S)$ ；
3. 加权聚合值向量： $O = PV$ 。

在 PyTorch 的朴素实现中，这三个步骤通常是独立的内核。这意味着中间结果 S 和 P 都要被写回到全局内存 (HBM)，下一步再读回来。当序列长度 N 很大时， S 和 P 的大小都是 $N \times N$ ，内存访问量随序列长度呈平方增长，注意力很快就从计算受限变成**内存受限**。

FlashAttention 的核心目标

FlashAttention 的目标不是“让矩阵乘法更快”，而是**减少从 HBM (全局内存) 搬运的数据量**。只要能把数据尽量留在更快的 SRAM (共享内存/寄存器) 里完成运算，就能显著提升端到端效率，并支持更长的上下文。

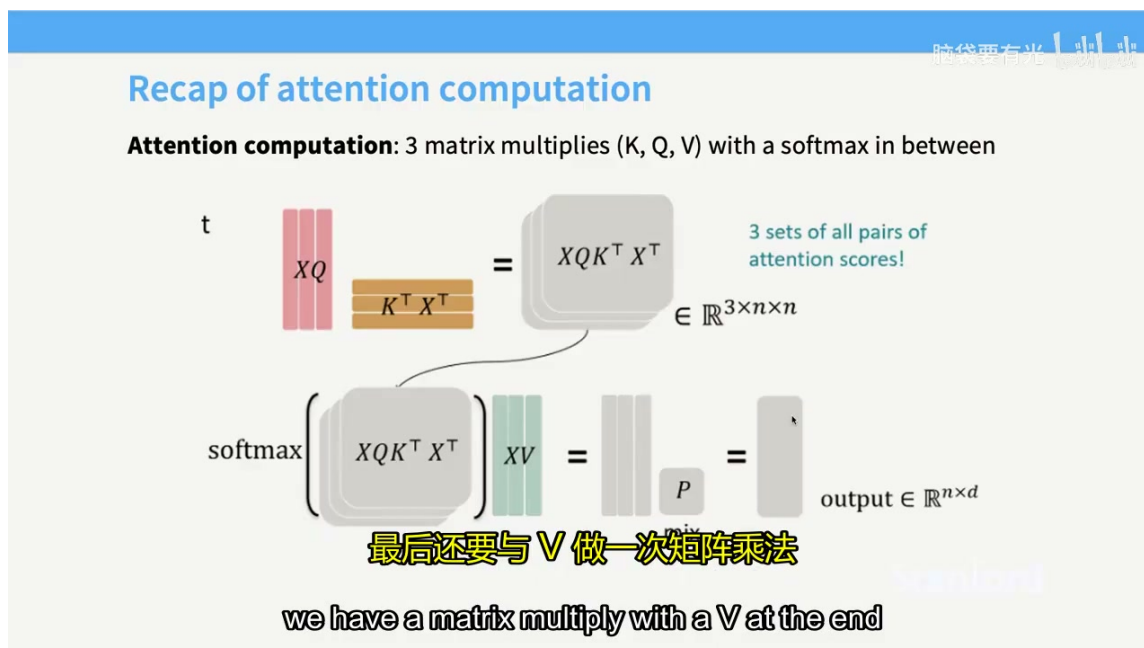


图 20: 标准 Attention 计算流程回顾: 依次执行 QK^T 矩阵乘法、softmax 归一化、再与 V 相乘得到输出。该图展示了讲座中对注意力三个矩阵乘加中间 softmax 的完整计算路径。²⁰

上图示意了标准 Attention 的数据流: QK^T 、Softmax、 PV 三个操作各自作为独立内核, 每一步都要与 HBM 交换完整的 $N \times N$ 矩阵。对于长序列而言, 这种反复读写全局内存的开销远超计算本身。

5.2 分块与在线 softmax

FlashAttention 的第一步是**分块** (tiling): 把 Q 、 K 、 V 都切分成适合 SRAM 的小块, 然后逐块完成矩阵乘法。讲座中指出, QK^T 和 PV 这两次矩阵乘法本身都可以通过分块实现, 这一点与第 4.7 节介绍的分块矩阵乘法完全一致。

真正的难点在于 softmax。softmax 是一个全局操作: 为了数值稳定, 必须先求出整行的最大值, 再用这个最大值对整行做归一化。如果我们把矩阵切成若干块, 各块之间似乎必须等全局统计完成后才能继续, 这就破坏了分块的流水线。

解决这个问题的关键是**在线 softmax** (online softmax)。它的思想是: 不必等所有数据都到齐, 而是随着新数据块的到来, 逐步更新当前行的最大值和归一化因子。

²⁰原视频时间戳: 01:13:15。

在线 softmax 更新规则

设当前已经处理了前若干块，已知旧最大值 m 、旧指数和 d 以及旧输出累加值 o 。当新的一块到来后，得到新最大值 m' 、新增指数和 d_{new} 以及新块的值加权和 v_{new} ，更新规则为：

$$m' = \max(m, m_{\text{new}}), \quad (1)$$

$$d' = e^{m-m'} \cdot d + e^{m_{\text{new}}-m'} \cdot d_{\text{new}}, \quad (2)$$

$$o' = \frac{d \cdot o \cdot e^{m-m'} + e^{m_{\text{new}}-m'} \cdot v_{\text{new}}}{d'}. \quad (3)$$

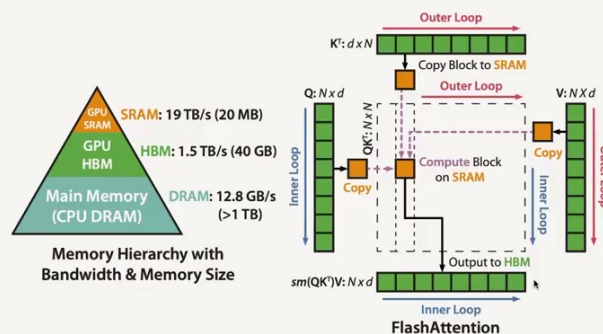
其中最关键的一步是**缩放因子修正**：一旦全局最大值被替换，之前所有累加的指数和 d 以及输出累加值 o 都要按 $e^{m-m'}$ 重新缩放，以保证与新块的统计量处于同一“量纲”。最终，所有块处理完毕后， o' 就是当前行的正确输出。

借助在线 softmax，FlashAttention 可以在 SRAM 中逐块推进：

1. 将 Q 的一块和 K 的一块加载到 SRAM；
2. 计算局部分数 $S_{ij} = Q_i K_j^\top$ ；
3. 用在线 softmax 更新当前行的最大值 m 与归一化因子 d ；
4. 继续下一块，重复上述过程；
5. 所有块处理完毕后，再用最终的分块 V 计算输出 O 。

整个过程中，需要持久化到 HBM 的只是一个很小的状态（每行的最大值和归一化因子），而不是完整的 $N \times N$ 注意力矩阵。讲座中形象地描述为：虚线框表示在 SRAM 中分块处理，蓝色框表示 HBM 中的全局内存——FlashAttention 尽量让虚线框里的工作独立完成，只在必要时与蓝色框交换数据。

Tiling part 1: tiling for the KQV matrix multiply



This figure 1 from the paper is literally just tiling for a KQV matrix multiply..

论文的图一其实就是分块矩阵乘法
But what do we do about the softmax?
Figure one from the paper is
literally just tiled matrix multiplies.

图 21: FlashAttention 的分块矩阵乘法与存储层次：左侧为 GPU SRAM / HBM / CPU DRAM 的带宽—容量金字塔，右侧展示 FlashAttention 如何通过 Outer/Inner Loop 将 K^T 、 Q 、 V 的块拷贝到 SRAM，在片上完成分块计算后再将结果写回 HBM。²¹

在线 softmax 的数值稳定

在线 softmax 之所以可行，正是因为 softmax 的数值稳定形式天然允许“延迟归一化”。标准形式

$$\text{softmax}(x_i) = \frac{e^{x_i - \max_j x_j}}{\sum_k e^{x_k - \max_j x_j}}$$

中的最大值起到了“公共偏移量”的作用；在线更新时，只要把所有历史累加值统一缩放到新最大值下，就能保持等价性。

5.3 融合与重计算

除了分块和在线 softmax，FlashAttention 还充分利用了第 4.4 节和第 4.5 节的另外两项技巧：算子融合与重计算。

算子融合体现在： QK^T 、指数计算、在线 softmax 更新、与 V 的加权乘法，以及最后的归一化除法，被尽可能地合并到同一个自定义 CUDA 内核中。这样中间结果始终留在寄存器和共享内存里，避免了多次写入和读取 HBM。讲座中总结为“尽可能把这两步合并执行，避免来回搬运数据”。

重计算则体现在反向传播。如果我们在前向传播中保存完整的注意力矩阵 P （或等价的 S ），反向时仍然需要一个平方量级的激活值。FlashAttention 选择在前向时只保存每行的最大值 m 和归一化因子 d ——这两个量都是 $O(N)$ 的——然后在反向传播时**逐块重新计算**所需的注意力分数和 softmax 结果。虽然这增加了一定量的重复计算，但它把内存占用从 $O(N^2)$ 降到了 $O(N)$ ，同时由于重计算过程本身也是分块融合的，额外的计算成本远小于节省的内存访问成本。

²¹原视频时间戳：01:13:30。

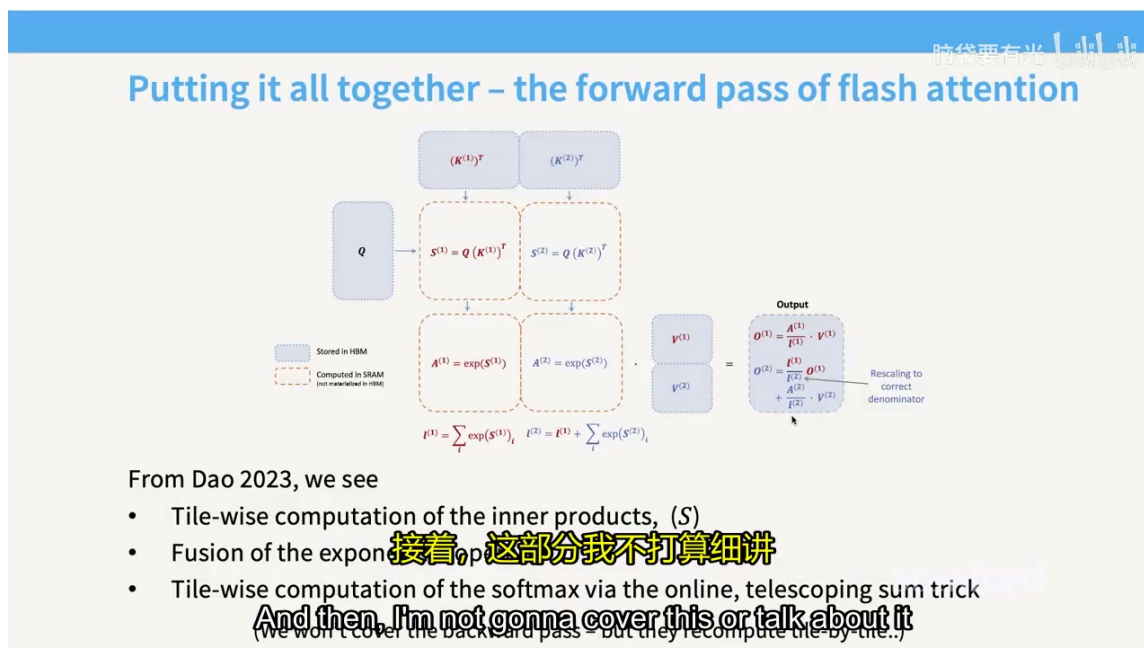


图 22: FlashAttention 正向数据流: 图中虚线框表示在 SRAM 中计算且不在 HBM 物化的中间结果 (S 、 A 、部分累加和), 实线框为存储在 HBM 的 Q 、 K 、 V 与最终输出。通过在线 softmax 的缩放技巧, 输出按块逐步更新并做分母校正; 反向传播时则按块重计算这些中间值, 而非保存完整的 $N \times N$ 激活矩阵。²²

上图对比了 FlashAttention 前向与反向的数据流: 前向通过分块 + 在线 softmax 在 SRAM 中完成; 反向则利用保存的 m 、 d 和输入 Q 、 K 、 V 逐块重算, 同样避免将巨大的 $N \times N$ 矩阵写回 HBM。

FlashAttention 的三根支柱

- **分块 (Tiling)**: 把 Q 、 K 、 V 切成适合 SRAM 的小块, 降低全局内存访问量。
- **在线 softmax (Online Softmax)**: 逐块更新最大值与归一化因子, 避免全局同步。
- **重计算 (Recomputation)**: 反向传播时逐块重新计算注意力分数, 将激活内存从 $O(N^2)$ 降到 $O(N)$ 。

这三者的组合, 使得 FlashAttention 成为一个计算高效、内存可扩展的融合内核。

5.4 本章小结

FlashAttention 是本章乃至整个系统模块的最佳综合案例。它没有改变注意力的数学公式, 却通过深入理解 GPU 的存储层次和执行模型, 把一个内存受限的操作改造成了计算密集的融合内核。

回顾讲座强调的六个技巧:

- 分块 (tiling) 让大数据集适配有限的 SRAM;
- 算子融合 (fusion) 减少了 HBM 的反复读写;
- 重计算 (recomputation) 用额外的计算换取内存和带宽;

²²原视频时间戳: 01:16:48。

- 低精度、合并访问、控制发散等技巧也可以在 FlashAttention 的底层实现中继续发挥作用。

正如主讲人所言，当前算力增长的方式要求我们把重点放在**算术密度极高的核心操作**上。理解硬件——寄存器、共享内存、L2、HBM 的层次，SM、Warp、线程块的调度，以及 GPU 与 TPU 的趋同演化——是设计高效模型和系统的前提。只有把这些底层直觉内化，才能在面对新的模型结构或硬件平台时，做出有理有据的系统决策。

6 总结与延伸

6.1 主讲人的 closing message

讲座接近尾声时，主讲人重新回到这堂课的初心：所有看似复杂的 GPU 优化技巧，归根结底都来自同一套核心思想。他指出，FlashAttention 之所以重要，不仅因为它把注意力的内存复杂度从 $O(N^2)$ 降到了 $O(N)$ ，更因为它同时运用了分块、在线 softmax、算子融合与重计算等多项技巧——我们在这节课里看到的各种高效实现，本质上都是这些思想的不同组合。

主讲人特别强调，理解硬件至关重要。GPU 是推动规模化计算的核心，我们不能再把底层细节当作黑箱；只有深入理解 SM、Warp、存储层次和数据移动，才能明白为什么要做这些选择，而不是盲目照搬某个固定配置。他也不希望学生成为“照搬 32 个乘法器来设计矩阵”的人，而是要真正理解这些设计决策背后的原因。

最后，他总结了未来系统设计的两个关键词：**算术密度极高的核心操作与数据移动**。在当前计算能力的增长方式下，算力与内存之间的差距越来越大；谁能减少数据在 HBM 与 SRAM 之间的往返，谁就能把理论峰值转化为实际性能。

6.2 本讲核心要点回顾

通过这五个章节，我们建立了从硬件到算法的完整直觉链：

1. **后登纳德时代的算力增长靠并行**。CPU 主频已触顶，GPU/TPU 通过横向扩展大量轻量计算单元，持续推高吞吐量。
2. **GPU 的核心是 SM + 层次化存储**。寄存器最快、SRAM/L1 次之、L2 再次、HBM 最慢。高效程序要尽量让数据停留在靠近计算单元的地方。
3. **GPU 与 TPU 殊途同归**。两者都拥有矩阵乘法单元、海量并行和层次化存储，只是 GPU 用大量小型 Tensor Core 换灵活性，TPU 用少量大型 MXU 换专用效率。
4. **六个优化技巧本质都是减少内存移动**。控制发散、低精度、算子融合、重计算、合并访问、分块与波量化——前五项直接减少数据搬运或让每次搬运承载更多计算，分块则利用 SRAM 复用把全局内存访问降到最低。
5. **FlashAttention 是这些思想的集大成者**。通过分块 + 在线 softmax，它在 SRAM 内完成注意力的核心计算；通过重计算，它把激活内存从 $O(N^2)$ 降到 $O(N)$ ；通过融合，它避免了中间结果反复写回 HBM。

一个贯穿始终的公式

$$\text{有效性能} = \text{峰值算力} \times \text{利用率}$$

峰值算力由硬件决定，利用率则由**数据移动**、**分支发散**、**精度格式**、**分块对齐**等因素共同决定。系统优化的本质，就是让利用率尽可能接近 1。

6.3 延伸思考

- **从训练到推理**：本讲主要围绕训练场景中的算子优化展开，但这些思想在推理场景中同样关键。量化（INT8/FP8/FP4）、算子融合和 FlashAttention 的变体（如 FlashDecoding、FlashInfer）正在重塑大模型推理的延迟与吞吐曲线。
- **从 GPU 到专用芯片**：随着模型规模继续扩大，TPU、Groq、SambaNova 等专用加速器会越来越多。理解 GPU 的优化逻辑，能帮助我们快速迁移到新的硬件抽象上——因为高效计算的基本原则不变。
- **从手工内核到编译器**：`torch.compile`、`jax.jit`、Triton 等工具正在把许多优化自动化，但编译器仍无法解决所有问题。对于注意力、MoE 路由、自定义损失函数等核心算子，手写 CUDA 或 Triton 内核仍然是获得极致性能的关键。

推荐阅读

- *Programming Massively Parallel Processors* (Štěpán Doštál 等) ——主讲人推荐的 GPU 入门教材；
- Horace He 的博客与 `gpu-mode` 社区——关于 GPU 底层机制与 PyTorch 内核优化的实践资源；
- FlashAttention 原始论文 (Dao et al., 2022) ——将本讲思想工程化的经典实现。